

Cours 3

1. Table de hachage
2. Dictionnaires en Python

Table de hachage

- Une **table de hachage (ou dictionnaire, ou table d'association)**, est une structure de données qui permet une association clé–valeur, c'est-à-dire une implémentation du type abstrait tableau associatif.
- Le but principal est de permettre de retrouver une clé donnée très rapidement, en la cherchant à un emplacement de la table correspondant au résultat d'une fonction de hachage calculée en $O(1)$.
 - un gain de temps très important pour les grosses tables,
 - une recherche ou un besoin d'accès aux données en utilisant la clé définie.

Exemple d'utilisation :

- Construction d'un annuaire téléphonique qui permet de stocker des couples
[nom ; numéro de téléphone]

Formalisation du problème

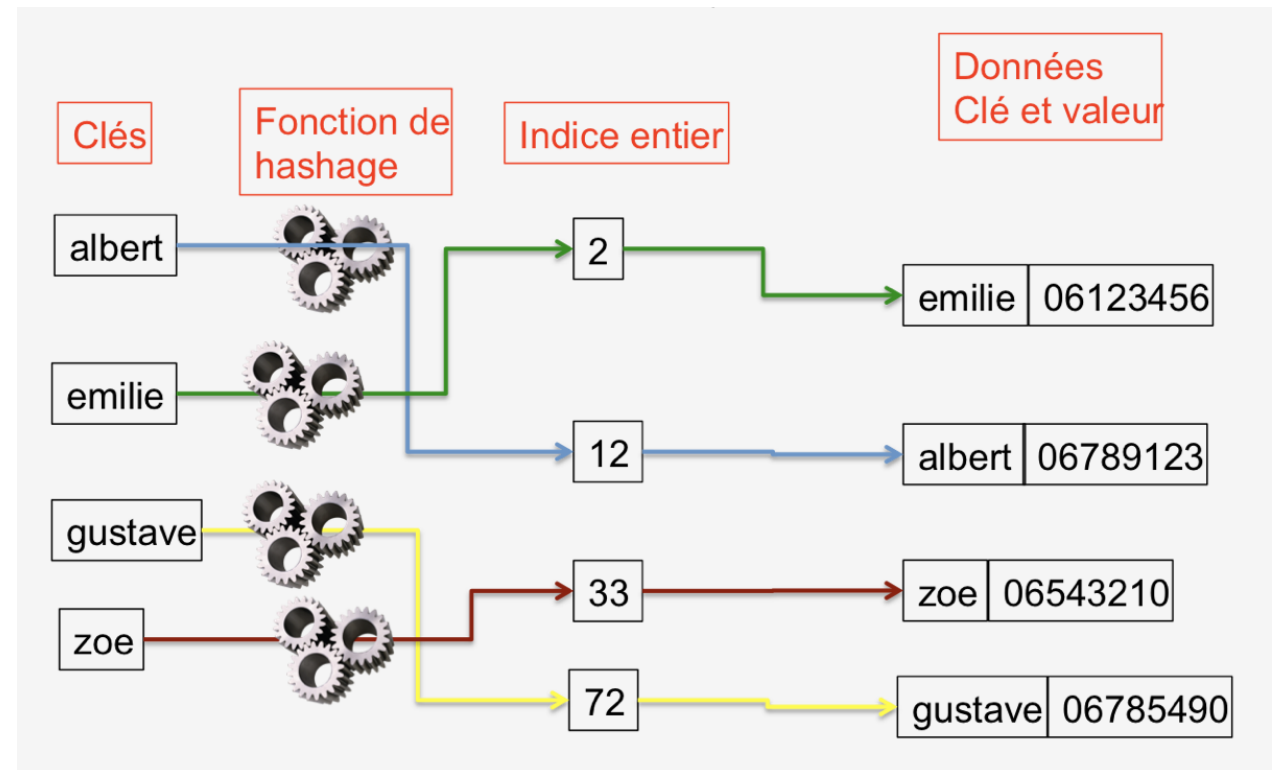
- Nous cherchons à indexer un ensemble de valeurs (les numéros de téléphones) par un ensemble de clés (les noms).
 - Ce n'est pas possible avec un tableau car les indices sont obligatoirement des entiers.
- Les **clés** peuvent être n'importe quel objet informatique.
 - Les clés sont des chaînes de caractères : les noms.
- Les **valeurs** sont aussi n'importe quel objet informatique.
 - Ce sont les données associées à la clé.
 - Les valeurs sont les numéros de téléphone.
- Comment rester efficace lorsque l'on recherche les valeurs associées à une clé ?
 - La structure la plus rapide reste le tableau.
 - Il faut trouver un moyen de transformer une clé en un entier.

Principe des tables de hachage

- Le premier principe essentiel des tables de hachage est de transformer une clé en un entier, qui sera l'indice du tableau contenant les données.
- **La fonction de hachage** est une fonction qui calcule un indice entier à partir d'une clé quelconque.
- La fonction de hachage assure une correspondance entre la clé et l'indice entier du tableau.

Construction d'un annuaire

- Pour annuaire comportant les numéros de téléphones, il faut mettre en œuvre :
 - une fonction de hachage qui transforme une clé (un nom) en un indice entier
 - un tableau de structures contenant la clé (le nom) et la valeur (le numéro de téléphone) associées à cette clé



Ajouter un couple [clé,valeur] dans une table d'association

- Nous souhaitons construire un dictionnaire de la langue française.
- Nous définissons :
 - un tableau de M structures contenant deux champs : une chaîne de caractères pour la clé (le mot) et une chaîne de caractères pour la valeur (sa définition),
 - une fonction de hachage qui transforme une clé (un mot de la langue française) en un indice entier.

Exemple de la fonction de hachage :

$\text{hash}(\text{mot}) = (\text{somme des lettres du mot, avec la correspondance 'a' } \rightarrow 1, \text{'b' } \rightarrow 2, \dots \text{'z' } \rightarrow 26) \text{ modulo } M$

Exemple : ajout dans une table d'association

$\text{hash}(\text{mot}) = (\text{somme des lettres du mot, avec la correspondance 'a' } \rightarrow 1, \text{'b' } \rightarrow 2, \dots \text{'z' } \rightarrow 26) \text{ modulo } M$

Exemple :

- on calcule l'indice de hachage

$\text{hash}(\text{"lame"}) = (12 + 1 + 13 + 5) \% 100 = 31$

- on remplit la case d'indice 31 de la table avec la clé « lame » et sa définition

	Clé	Valeur
0		
1		
....		
6	bac	examen passé en terminale
...		
31	lame	partie coupante d'un couteau
...		
99		

Ajout dans une table d'association

Pour **ajouter** un couple [clé, valeur] à une table d'association, il faut donc :

- calculer l'indice que ce couple occupera dans la table, c'est à dire exécuter la fonction de hachage pour récupérer la valeur entière qu'elle retourne ;
- placer ce couple [clé, valeur] dans la table à l'indice qui vient d'être calculé.

	Clé	Valeur
0		
1		
....		
6	bac	examen passé en terminale
...		
31	lame	partie coupante d'un couteau
...		
99		

Recherche dans une table d'association

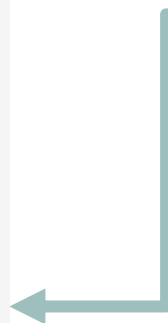
Pour **rechercher** une clé et/ou sa valeur dans une table d'association, il faut :

- calculer l'indice que ce couple devrait occuper dans la table, à l'aide de la fonction de hachage ;
- s'ils existent, la clé et la valeur sont dans la table, à l'indice qui vient d'être calculé.

	Clé	Valeur
0		
1		
...		
6	bac	examen passé en terminale
...		
31	lame	partie coupante d'un couteau
...		
99		

On cherche la définition d'un mot "lame":

$\text{hash}(\text{"lame"}) = (12 + 1 + 13 + 5) \% 100 = 31$



Propriétés des fonctions de hachage

Dans une table de hachage, la fonction de hachage est utilisée entre autres chaque fois que l'on souhaite ajouter ou rechercher une clé dans la table.

Elle doit posséder **2 propriétés essentielles** :

- la **rapidité** de calcul
 - si le temps de calcul de la fonction de hachage est trop long, autant utiliser un tableau tout simple.
- la **dispersion**
 - la fonction de hachage doit répartir le mieux possible toutes les valeurs sur l'ensemble des entiers autorisés ($0..M-1$).

Hachage et collisions

- Que se passe-t-il lorsque l'on veut ajouter le mot **male** et sa définition ?
- L'indice de hachage :
 $\text{hash}(\text{"male"}) = (13+1+15+5)\%100 = 31$
- Il faudrait mettre le mot **male** à l'indice 31 dans la table.
- **Mais : cet indice est déjà occupé par un autre couple (clé, définition), celle du mot lame !**
- C'est ce qu'on appelle une **collision**.

	Clé	Valeur
0		
1		
....		
6	bac	examen passé en terminale
...		
31	lame	partie coupante d'un couteau
...		
99		

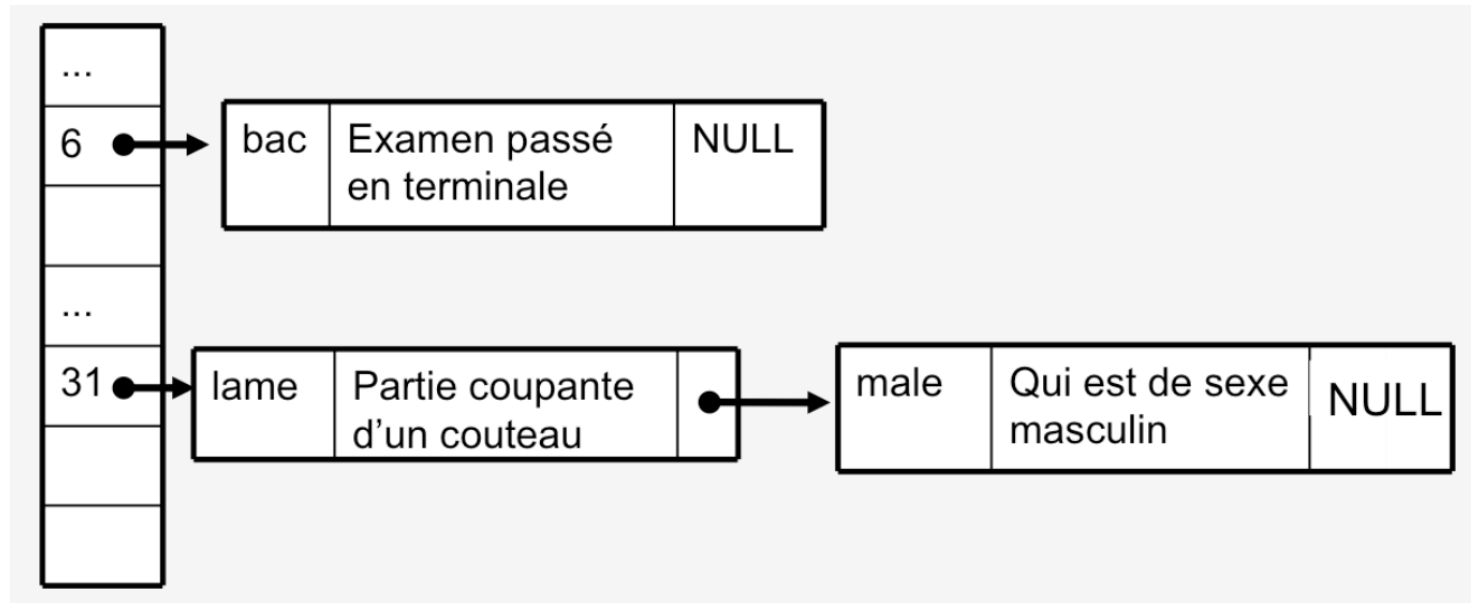
Comment gérer les collisions ?

Il existe 2 grandes familles de solutions pour gérer les collisions :

- Le hachage par chaînage
- Le hachage ouvert

Le hachage par chaînage

- La table sera un tableau de listes chaînées (voir CM4-CM7).
- Dans les M listes de la table, chaque maillon contient une clé et une définition, ainsi que l'adresse du maillon suivant.
- Pour ajouter un couple (clé,valeur) à notre table, il suffit d'ajouter ce couple dans un nouveau maillon, en tête de la liste située à l'indice de hachage.



Le hachage ouvert

- La table est un tableau de M structures avec un champ `cle` et un champ `valeur`.
- Si une collision apparaît, on recherche une case **vide** parmi les cases suivantes dans notre table à l'aide d'une fonction de sondage.
- La fonction de sondage la plus simple est de choisir la case suivante, puis la suivante, etc... jusqu'à trouver une case **libre**.

	Clé	Valeur
0		
...		
6	bac	examen passé en terminale
...		
31	lame	partie coupante d'un couteau
32	male	De sexe masculin
...		
...		
99		

Déclaration

- On suppose que les types `key_t` et `value_t` désignent respectivement le type des clés et le type des valeurs.
- On suppose également que le type `element_t` est une structure avec les 2 champs contenant les valeurs et clés

`element_t : structure`

`key : key_t                      /* clé */`

`value : value_t    /* valeur de clé */`

`finstructure`

Méthodes de table de hachage

Méthode	Description
<code>hashtable_new(m: entier) : hashtable_t</code>	crée et retourne une nouvelle table de <code>m</code> listes vides
<code>hashtable_put(k: key_t, v:value_t, t: hashtable_t): entier</code>	ajoute le couple [clé, valeur] à la table <code>t</code> . Retourne 1 si réussi, 0 sinon ;
<code>hashtable_contains(k: key_t , t, hashtable_t): entier</code>	retourne 1 (true) si la clé <code>k</code> existe dans la table, 0 sinon.
<code>hashtable_getvalue(k: key_t , pv: value_t, t: hashtable_t): entier</code>	Recherche la clé <code>k</code> dans la table. Si la clé est présente, remplit la variable <code>pv</code> avec la valeur associée à la clé et retourne 1. Si la clé n'est pas présente, retourne 0.
<code>hashtable_delete_key(k: key_t, t: hashtable_t): entier</code>	Supprime la clé <code>k</code> et la valeur associée de la table <code>t</code> retourne 1 si la suppression est réalisée, 0 sinon
<code>hashtable_delete(t: hashtable_t) : hashtable_t</code>	Supprime tous les éléments, détruit la table et retourne une table vide
<code>hashtable_print(t: hashtable_t)</code>	Affiche les éléments de la table.

Implémentations de ces méthodes

- Hachage par chaînage
 - La table de hachage est une table de listes chaînées, chaque maillon contenant la clé et la valeur associée.
 - On utilise les fonctions de base sur les listes chaînées pour ajouter en tête, rechercher si une clé est déjà présente, obtenir la valeur associée à une clé, supprimer un couple [clé, valeur].

Implémentations de ces méthodes

- Hachage par chaînage
 - La table de hachage est une table de listes chaînées, chaque maillon contenant la clé et la valeur associée.
 - On utilise les fonctions de base sur les listes pour ajouter en tête, rechercher si une clé est déjà présente, obtenir la valeur associée à une clé, supprimer un couple [clé, valeur].
- Hachage ouvert
 - Le hachage ouvert est une alternative dans laquelle tous les éléments sont stockés dans un tableau, sans listes chaînées.
 - Il ne peut fonctionner que si le nombre d'éléments de la table est inférieur à la taille de la table.

Principes du Hachage ouvert et sondage

Pour insérer un couple [clé k , valeur v], on sonde les cases à partir de la case donnée par la fonction de hachage: on cherche la première case vide à partir de celle d'indice $\text{hash}(k)$.

Il existe plusieurs stratégies de sondage :

- **sondage linéaire** : à partir de l'indice $j = \text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+2$, ... jusqu'à trouver une case vide. La fonction de hachage totale s'écrit donc $j = \text{hash}(k) + i$ où i est le nombre d'essais.

Principes du Hachage ouvert et sondage

Pour insérer un couple [clé k , valeur v], on sonde les cases à partir de la case donnée par la fonction de hachage: on cherche la première case vide à partir de celle d'indice $\text{hash}(k)$.

Il existe plusieurs stratégies de sondage :

- **sondage linéaire** : à partir de l'indice $j = \text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+2$, ... jusqu'à trouver une case vide. La fonction de hachage totale s'écrit donc $j = \text{hash}(k) + i$ où i est le nombre d'essais.
- **sondage quadratique** : à partir de l'indice $j = \text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+4$, puis $j+9$... jusqu'à trouver une case vide. La fonction de hachage totale s'écrit donc $j = \text{hash}(k) + c_1 * i + c_2 * i * i$ où j est le nombre d'essais et c_1, c_2 sont des constantes.

Principes du Hachage ouvert et sondage

Pour insérer un couple [clé k , valeur v], on sonde les cases à partir de la case donnée par la fonction de hachage: on cherche la première case vide à partir de celle d'indice $\text{hash}(k)$.

Il existe plusieurs stratégies de sondage :

- **sondage linéaire** : à partir de l'indice $j = \text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+2$, ... jusqu'à trouver une case **vide**. La fonction de hachage totale s'écrit donc $j = \text{hash}(k) + i$ où i est le nombre d'essais.
- **sondage quadratique** : à partir de l'indice $j = \text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+4$, puis $j+9$... jusqu'à trouver une case **vide**. La fonction de hachage totale s'écrit donc $j = \text{hash}(k) + c_1 * i + c_2 * i * i$ où j est le nombre d'essais et c_1, c_2 sont des constantes.
- **sondage double hachage** : on utilise une deuxième fonction de hachage g . A partir de l'indice $j = \text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+g(k)$, puis $j+2 * g(k)$, ... jusqu'à trouver une case **vide**. La fonction de hachage totale s'écrit donc $j = \text{hash}(k) + i * g(k)$ où i est le nombre d'essais.

Principes du Hachage ouvert et sondage

Pour insérer un couple [clé k , valeur v], on sonde les cases à partir de la case donnée par la fonction de hachage: on cherche la première case vide à partir de celle d'indice $\text{hash}(k)$.

Il existe plusieurs stratégies de sondage :

- **sondage linéaire** : à partir de l'indice $j=\text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+2$, ... jusqu'à trouver une case **vide**. La fonction de hachage totale s'écrit donc $j=\text{hash}(k)+i$ où i est le nombre d'essais.
- **sondage quadratique** : à partir de l'indice $j=\text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+1$, puis $j+4$, puis $j+9$... jusqu'à trouver une case **vide**. La fonction de hachage totale s'écrit donc $j=\text{hash}(k)+c_1*i+c_2*i*i$ où j est le nombre d'essais et c_1, c_2 sont des constantes.
- **sondage double hachage** : on utilise une deuxième fonction de hachage g . A partir de l'indice $j=\text{hash}(k)$, on regarde si la case d'indice j est vide, puis la case $j+g(k)$, puis $j+2*g(k)$, ... jusqu'à trouver une case **vide**. La fonction de hachage totale s'écrit donc $j=\text{hash}(k)+i*g(k)$ où i est le nombre d'essais.

Insertion et hachage ouvert

// On veut ajouter les clés k= lame, male, mela, puis mena dans la table.

i=0

répéter

h = hash(k) + i

si la case h est libre

alors

remplir cette case avec la clé et sa valeur

sinon

incrémenter i

jusqu'à atteindre la fin de la table

Erreur car table pleine

	Clé	Valeur
0	VIDE	
...	VIDE	
...	VIDE	
...	VIDE	
31	lame	Partie coupante d'un couteau
32	male	De sexe masculin
33	mela	Fete traditionnelle indienne
34	mena	Passé simple du verbe mener
...	VIDE	
...	VIDE	
M-1	VIDE	

$\text{hash}(\text{"lame"})=31 \Rightarrow i=0$

$\text{hash}(\text{"male"})=31 \Rightarrow i=1$

$\text{hash}(\text{"mela"})=31 \Rightarrow i=2$

$\text{hash}(\text{"mena"})=33 \Rightarrow i=1$

Recherche et hachage ouvert

$i=0$

l = calculer l'indice de hachage de la clé k

répéter

$h = l + i$

si (la case h n'est pas vide) ET (si la clé k est égale à la clé stockée dans la table)

alors

retourner la valeur associée à la clé dans la case h

sinon

incrémenter i

jusqu'à atteindre une case VIDE ou la fin de la table

retourner "PAS TROUVE"

Suppression et hachage ouvert

- On ne peut pas simplement marquer à **VIDE** la case du tableau qui contenait la clé.
- Si on se contente de vider la case contenant la clé, alors l'algorithme d'insertion et l'algorithme de recherche ne fonctionnent plus.

Exemple :

- On a ajouté les entrées **lame**, **male**, **mela**, **mena** dans la table.
- Les cases d'indice **31 à 34** sont remplies.
- Si on supprime l'entrée **male** avec **l'algorithme de suppression erroné** qui se contente de vider la case (la case d'indice 32).

	Clé	Valeur
0	VIDE	
...	VIDE	
...	VIDE	
...	VIDE	
31	lame	Partie coupante d'un couteau
32	male	De sexe masculin
33	mela	Fete traditionnelle indienne
34	mena	Passé simple du verbe mener
...	VIDE	
...	VIDE	
M-1	VIDE	

Suppression et hachage ouvert

- On ne peut pas simplement marquer à **VIDE** la case du tableau qui contenait la clé.
- Si on se contente de vider la case contenant la clé, alors l'algorithme d'insertion et l'algorithme de recherche ne fonctionnent plus.

Exemple :

- On a ajouté les entrées **lame**, **male**, **mela**, **mena** dans la table.
- Les cases d'indice **31 à 34** sont remplies.
- Si on supprime l'entrée **male** avec **l'algorithme de suppression erroné** qui se contente de vider la case (la case d'indice 32).
- A partir de cet état, nous n'arriverons plus trouver le mot **mela** avec notre algorithme de recherche.

	Clé	Valeur
0	VIDE	
...	VIDE	
...	VIDE	
...	VIDE	
31	lame	Partie coupante d'un couteau
32	VIDE	
33	mela	Fete traditionnelle indienne
34	mena	Passé simple du verbe mener
...	VIDE	
...	VIDE	
M-1	VIDE	

Solution pour la suppression

Une solution : dans l'algorithme de suppression, de ne pas vider la case qui contenait la clé supprimée, mais d'indiquer que cette case contenait une clé qui a été supprimée.

On définit une clé spéciale SUPPRIME pour marquer les cases qui sont supprimées.

- cette clé est utilisée :
 - pour la suppression, on marque la case de la table qui contenait la clé supprimée avec la clé spéciale SUPPRIME.
 - pour l'insertion, si on trouve une clé SUPPRIME, on considère cette case comme disponible et on la remplit avec la clé *k* et la valeur associée.
 - pour la recherche, si on trouve une clé SUPPRIME lors de la recherche, on considère cette case comme occupée, et on passe à l'itération suivante pour le sondage.

Nouvel algorithme d'insertion

// Algorithme corrigé pour l'insertion de la clé k et de sa valeur

i=0

répéter

h = hash(k) + i

si la case h est VIDE ou SUPPRIME

alors

remplir cette case avec la clé et sa valeur

sinon

incrémenter i

jusqu'à atteindre la fin de la table

Erreur car table pleine

Nouvel algorithme de recherche

// Algorithme corrigé pour la recherche de la clé k

i=0

répéter

h = hash(k) + i

si la case h n'est pas VIDE

si (la case h est SUPPRIME) OU (si la clé k n'est pas égale à la clé à l'indice h)

alors

incrémenter i

sinon

retourner la valeur à cet indice h

jusqu'à la case h est VIDE ou la fin de la table est atteinte

retourner PAS TROUVE

Performance/Complexité

Les performances dépendent de 2 paramètres :

- la taille de la table M
- le nombre de clés utilisées N

On définit aussi le facteur de charge par $\alpha = N/M$

Remarque : dans le cas du chaînage linéaire, si $M \gg N$, $\alpha < 1$

- Si la fonction de hachage est satisfaisante (peu de collisions), alors toutes les opérations apparaîtront à coût constant $O(1)$... même s'il y a un très grand nombre (des millions, des milliards...) d'éléments

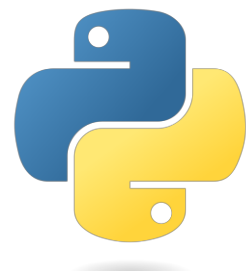


Tableau associatif (1/2)

- Le type `dict` de Python est une implémentation du type abstrait tableau associatif.
- L'implémentation correspond à une **table de hachage**, ce qui signifie que la valeur est stockée dans un tableau et que la position dans ce tableau dépend du résultat d'une fonction de hachage appliquée à la clé.
- En un temps indépendant du nombre de valeurs stockées dans le dictionnaire, Python peut retrouver la valeur associée à n'importe quelle clé : pour cela il calcule un indice à partir de la valeur de la clé (qui doit donc être hachable, c'est-à-dire récursivement non-mutable) et récupère la valeur stockée à cet indice dans un tableau.

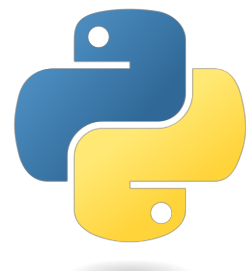
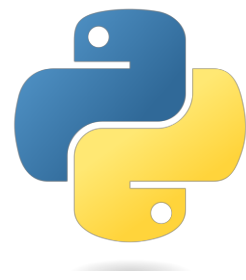


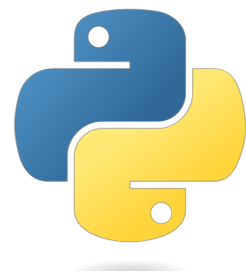
Tableau associatif (2/2)

- Une caractéristique essentielle des dictionnaires est que la récupération d'une valeur associée à une clé se fait en un temps constant, indépendant de la taille du dictionnaire.
- Savoir si une clé fait partie du dictionnaire prend un temps constant.
 - Vérifier si un élément est dans une liste prend un temps proportionnel à la taille de la liste.



Définition d'un dictionnaire

- Comme une liste, un **dictionnaire** est une collection mutable (modifiable) de plusieurs valeurs.
- Un **dictionnaire** en Python va aussi permettre de rassembler des éléments mais ceux-ci seront identifiés par **une clé**.
- Les index (les clés) des dictionnaires peuvent utiliser de nombreux types de données différents, et pas seulement des entiers.
- Une clé avec sa valeur associée est appelée paire **clé-valeur**.



Dictionnaires en Python (1/2)

- Les dictionnaires sont délimités par des accolades :

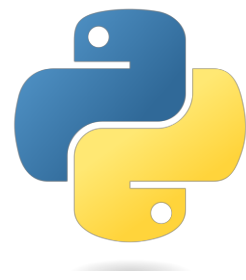
```
mon_dictionnaire = {"voiture": "véhicule à quatre roues",  
                    "vélo": "véhicule à deux roues"}
```

- Un élément a été défini dans le dictionnaire en précisant **une clé** au moyen d'une chaîne de caractères suivis de **:** puis de **la valeur associée**

clé: valeur

- On accède à une valeur du dictionnaire en utilisant la clé entourée par des crochets avec la syntaxe suivante :

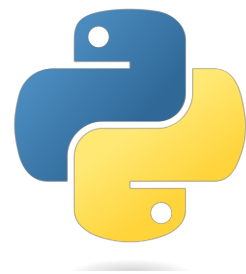
```
mon_dictionnaire["voiture"] ⇒ 'véhicule à quatre roues'
```



Dictionnaires en Python (2/2)

- Les dictionnaires peuvent aussi utiliser des valeurs entières comme **clés**
 - tout comme les listes utilisent des entiers comme index
 - mais ils ne doivent pas nécessairement commencer à 0 et peuvent être n'importe quel nombre.

```
spam = {12345: 'Luggage nubmer', 42: 'The Answer'}
```



Création d'un dictionnaire

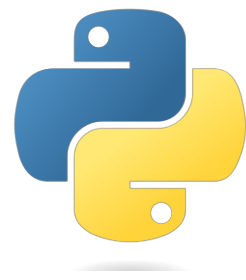
- Il est possible de créer un dictionnaire avec des accolades qui entourent les éléments.

```
mon_dictionnaire = {"voiture": "véhicule à quatre roues",  
                    "vélo": "véhicule à deux roues"}
```

- Une autre approche : créer un dictionnaire vide et à ajouter les éléments au fur et à mesure.

```
nombre_de_pneus = {}  
nombre_de_pneus["voiture"] = 4  
nombre_de_pneus["vélo"] = 2
```

```
nombre_de_pneus ⇒ {'voiture': 4, 'vélo': 2}
```



Dictionnaires vs. Lists (1/2)

- Les éléments des listes **sont ordonnés**.

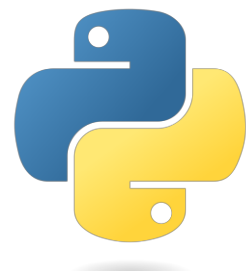
Exemple :

- Le premier élément d'une liste nommée `L1` serait `L1[0]`.

```
L1 = ['cats', 'dogs', 'moose']
```

```
L2 = ['dogs', 'moose', 'cats']
```

```
L1 == L2  $\Rightarrow$  False
```



Dictionnaires vs. Lists (2/2)

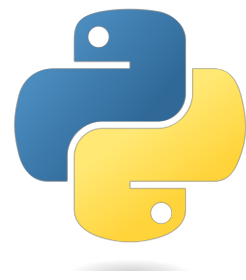
- Les éléments des dictionnaires **ne sont pas ordonnés**.
- Il n'y a pas de "premier" élément dans un dictionnaire.
- L'ordre dans lequel les paires clé-valeur sont saisies dans un dictionnaire n'a pas d'importance.

Exemple :

```
dict1 = {'name': 'Zophie', 'species': 'cat', 'age': '8'}
```

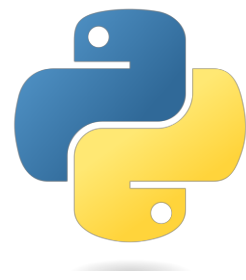
```
dict2 = {'species': 'cat', 'age': '8', 'name': 'Zophie'}
```

```
dict1 == dict2 ⇒ True
```



Exercice 1 :

- Définissez un dictionnaire avec les noms comme clés et les dates d'anniversaires comme valeurs.
- Créez un programme qui demande le nom de la personne à l'utilisateur.
- Si le nom est enregistré dans le dictionnaire, le programme donne la date d'anniversaire de cette personne.
- Sinon, le programme demande l'utilisateur de saisir la date d'anniversaire et ajoute le nom et la date d'anniversaire saisi dans le dictionnaire.



Exercise 1 : correction

```
birthdays = {'Alice': 'Apr 1', 'Bob': 'Dec 12', 'Carol': 'Mar 4'}
```

```
while True:
```

```
    print('Enter a name: (blank to quit)')
```

```
    name = input()
```

```
    if name == '':
```

```
        break
```

```
    if name in birthdays:
```

```
        print(birthdays[name] + ' is the birthday of ' + name)
```

```
    else:
```

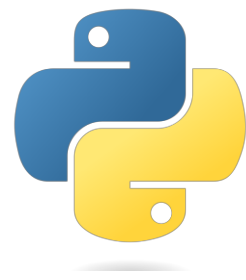
```
        print('I do not have birthday information for ' + name)
```

```
        print('What is their birthday?')
```

```
        bday = input()
```

```
        birthdays[name] = bday
```

```
        print('Birthday database updated.')
```



Exercise 1 : correction

Python Tutor: Visualize code in [Python](#), [JavaScript](#), [C](#), [C++](#), and [Java](#)

Python 3.6
[known limitations](#)

```
1 birthdays = {'Alice': 'Apr 1', 'Bob': 'Dec 12', 'Carol': 'Mar 4'}
2
3 while True:
4     print('Enter a name: (blank to quit)')
5     name = input()
6     if name == '':
7         break
8
9     if name in birthdays:
10        print(birthdays[name] + ' is the birthday')
11    else:
12        print('I do not have birthday information')
13        print('What is their birthday?')
14        bday = input()
15        birthdays[name] = bday
16        print('Birthday database updated.')
```

[Edit this code](#)

→ line that just executed

→ next line to execute



<< First < Prev Next > Last >>

Step 2 of 25

Print output (drag lower right corner to resize)



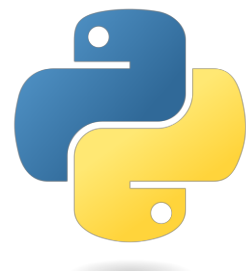
Frames

Objects

Global frame
birthdays

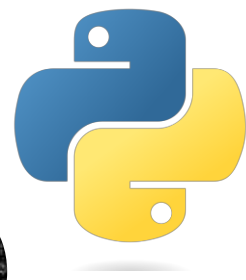
dict

"Alice"	"Apr 1"
"Bob"	"Dec 12"
"Carol"	"Mar 4"



Méthodes de parcours d'un dictionnaire

- Il existe trois méthodes qui renvoient des valeurs de type liste des clés, des valeurs ou des clés et des valeurs du dictionnaire :
 - `keys()`
 - `values()`
 - `items()`
- Les valeurs renvoyées par ces méthodes ne sont pas de vraies listes : elles ne peuvent pas être modifiées et n'ont pas de méthode `append()`.
- Mais ces types de données (`dict_keys`, `dict_values` et `dict_items`, respectivement) peuvent être utilisés dans des boucles `for`.



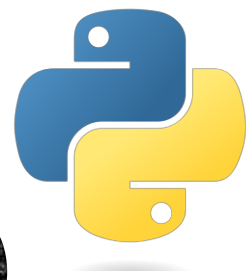
Parcours d'un dictionnaire (1/3)

- Pour parcourir un dictionnaire par les valeurs :

```
spam = {'color': 'red', 'age': 42}  
for v in spam.values():  
    print(v)
```

- Sur l'écran on aura :

```
red  
42
```



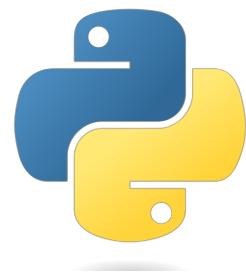
Parcours d'un dictionnaire (2/3)

- Pour parcourir un dictionnaire par les clés :

```
spam = {'color': 'red', 'age': 42}  
for k in spam.keys():  
    print(k)
```

- Sur l'écran on aura :

```
color  
age
```



Parcours d'un dictionnaire (3/3)

- Pour parcourir un dictionnaire par les clés et les valeurs, on utilise `items()`

```
spam = {'color': 'red', 'age': 42}
```

Une boucle `for` avec un indice `i`

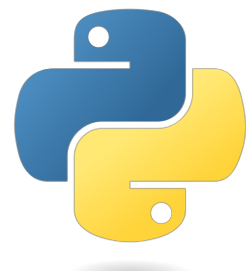
```
for i in spam.items():  
    print(i)
```

- Sur l'écran on aura :
('color', 'red')
('age', 42)

Une boucle `for` avec deux indices : `cle` et `valeur`

```
for cle, valeur in spam.items():  
    print("La clé", cle, "vaut", valeur)
```

- Sur l'écran on aura :
La clé color vaut red
La clé age vaut 42



Remarque : méthodes de parcours

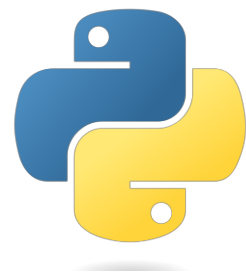
- Si vous souhaitez obtenir une « vrai » liste à partir des méthodes `keys()`, `values()`, `items()`, passez sa valeur de retour de cette liste à la fonction `list()`.

Exemple:

```
spam = {'color': 'red', 'age': 42}
```

```
spam.keys() ⇒ dict_keys(['color', 'age'])
```

```
list(spam.keys()) ⇒ ['color', 'age']
```



Vérifier l'existence d'une clé ou d'une valeur

- Vous pouvez utiliser les opérateurs `in` et `not in` pour vérifier si une certaine clé ou valeur existe dans un dictionnaire.

Exemple :

```
spam = {'name': 'Zophie', 'age': 7}
```

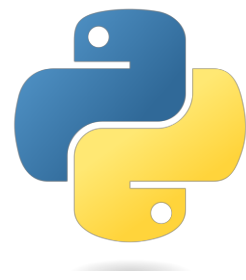
```
'name' in spam.keys() ⇒ ?
```

```
'Zophie' in spam.values() ⇒ ?
```

```
'color' in spam.keys() ⇒ ?
```

```
'color' not in spam.keys() ⇒ ?
```

```
'color' in spam ⇒ ?
```



Vérifier l'existence d'une clé ou d'une valeur

- Vous pouvez utiliser les opérateurs `in` et `not in` pour vérifier si une certaine clé ou valeur existe dans un dictionnaire.

Exemple :

```
spam = {'name': 'Zophie', 'age': 7}
```

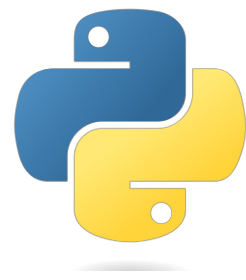
```
'name' in spam.keys() ⇒ True
```

```
'Zophie' in spam.values() ⇒ True
```

```
'color' in spam.keys() ⇒ False
```

```
'color' not in spam.keys() ⇒ True
```

```
'color' in spam ⇒ False
```



Méthode `get()`

- Il est fastidieux de vérifier si une clé existe dans un dictionnaire avant d'accéder à sa valeur.
- Les dictionnaires disposent d'une méthode `get()` qui prend deux arguments : la clé de la valeur à récupérer et une valeur de remplacement à renvoyer si cette clé n'existe pas.

Exemple :

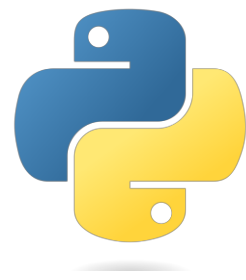
```
picnicItems = {'apples': 5, 'cups': 2}
```

```
'I am bringing ' + str(picnicItems['cups']) + ' cups.'
```

```
⇒ 'I am bringing 2 cups.'
```

```
'I am bringing ' + str(picnicItems['eggs']) + ' eggs.'
```

```
⇒ ?
```



Méthode `get()`

- Il est fastidieux de vérifier si une clé existe dans un dictionnaire avant d'accéder à sa valeur.
- Les dictionnaires disposent d'une méthode `get()` qui prend deux arguments : la clé de la valeur à récupérer et une valeur de remplacement à renvoyer si cette clé n'existe pas.

Exemple :

```
picnicItems = {'apples': 5, 'cups': 2}
```

```
'I am bringing ' + str(picnicItems['cups']) + ' cups.'
```

```
⇒ 'I am bringing 2 cups.'
```

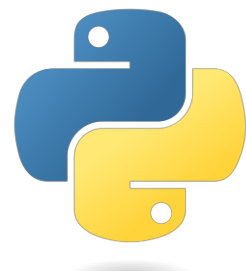
```
'I am bringing ' + str(picnicItems['eggs']) + ' eggs.'
```

```
Traceback (most recent call last):
```

```
File "<pyshell#34>", line 1, in <module>
```

```
'I am bringing ' + str(picnicItems['eggs']) + ' eggs.'
```

```
KeyError: 'eggs'
```



Méthode `get()`

- Il est fastidieux de vérifier si une clé existe dans un dictionnaire avant d'accéder à sa valeur.
- Les dictionnaires disposent d'une méthode `get()` qui prend deux arguments : la clé de la valeur à récupérer et une valeur de remplacement à renvoyer si cette clé n'existe pas.

Exemple :

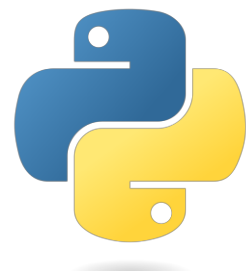
```
picnicItems = {'apples': 5, 'cups': 2}
```

```
'I am bringing ' + str(picnicItems.get('cups', 0)) + ' cups.'
```

```
⇒ 'I am bringing 2 cups.'
```

```
'I am bringing ' + str(picnicItems.get('eggs', 0)) + ' eggs.'
```

```
⇒ 'I am bringing 0 eggs.'
```



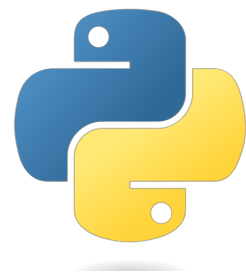
Méthode `setdefault()` (1/2)

- Vous pouvez définir une valeur dans un dictionnaire pour une certaine clé uniquement si cette clé n'a pas déjà une valeur.

Exemple :

```
dict = {'name': 'Polka', 'age': 3}
if 'color' not in dict:
    dict['color'] = 'black'
```

- La méthode `setdefault()` permet de le faire en une seule ligne de code.



Méthode `setdefault()` (2/2)

- Le premier argument transmis à la méthode est la clé à vérifier, et le second argument est la valeur à attribuer à cette clé si elle n'existe pas.
- Si la clé existe, la méthode `setdefault()` renvoie la valeur de la clé.

Exemple :

```
dict = {'name': 'Polka', 'age': 3}
```

```
dict.setdefault('color', 'black')
```

```
⇒ 'black'
```

```
dict
```

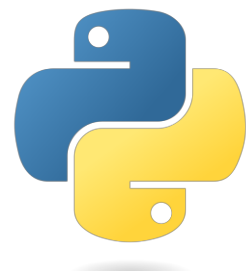
```
⇒ {'color': 'black', 'age': 3, 'name': 'Polka'}
```

```
dict.setdefault('color', 'white')
```

```
⇒ 'black'
```

```
dict
```

```
⇒ {'color': 'black', 'age': 5, 'name': 'Polka'}
```



Exercice 2 :

- Écrivez un programme qui compte le nombre d'occurrences de chaque lettre dans une chaîne de caractères.

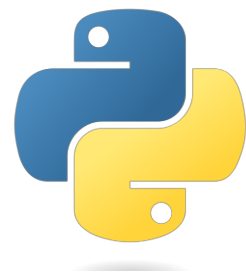
Exemple :

Entrée :

message = 'It was a bright cold day in April, and the clocks were striking thirteen.'

Sortie :

```
{ ' ': 13, ',': 1, '.': 1, 'A': 1, 'I': 1, 'a': 4, 'c': 3, 'b': 1, 'e': 5, 'd': 3, 'g': 2, 'i': 6, 'h': 3, 'k': 2, 'l': 3, 'o': 2, 'n': 4, 'p': 1, 's': 3, 'r': 5, 't': 6, 'w': 2, 'y': 1 }
```



Exercise 2 : correction

```
message = 'It was a bright cold day in April, and the clocks were striking thirteen.'
```

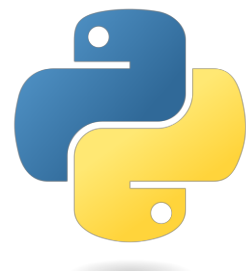
```
count = {}
```

```
for character in message:
```

```
    count.setdefault(character, 0)
```

```
    count[character] = count[character] + 1
```

```
print(count)
```



Exercise 2 : correction

Python Tutor: Visualize code in [Python](#), [JavaScript](#), [C](#), [C++](#), and [Java](#)

Python 3.6
[known limitations](#)

→ 1 message = 'It was a bright cold day in April, and t

2 count = {}

3

4 for character in message:

5 count.setdefault(character, 0)

6 count[character] = count[character] + 1

7

8 print(count)

[Edit this code](#)

→ line that just executed

→ next line to execute

<< First < Prev Next > Last >>

Step 1 of 223

If you use ChatGPT or other AI, [take this survey](#) **NEW:** Get AI Help

[Move and hide objects](#)

Print output (drag lower right corner to resize)

Frames Objects

[Ads keep this site free.](#) We are not responsible for contents of ads.

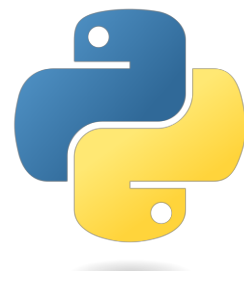
← Ads by Google

Stop seeing this ad

Why this ad? ▶

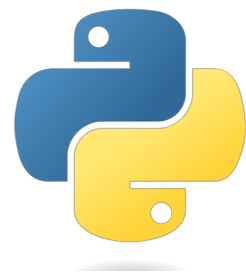
← Ads by Google

Stop seeing this ad Why this ad? ▶



Pretty Printing (1/2)

- Si vous importez le module `pprint` dans vos programmes, vous aurez accès aux fonctions `pprint()` et `pformat()` qui permettent d'imprimer les valeurs d'un dictionnaire.
- Cette fonction est utile lorsque vous souhaitez un affichage plus propre des éléments d'un dictionnaire que celui fourni par la fonction `print()`.



Pretty Printing (2/2)

Exemple :

```
import pprint
```

```
message = 'It was a bright cold day in April, and the clocks were striking thirteen.'
```

```
count = {}
```

```
for character in message:
```

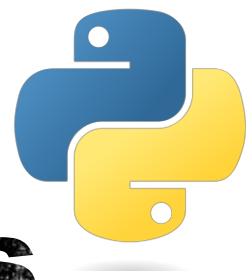
```
    count.setdefault(character, 0)
```

```
    count[character] = count[character] + 1
```

```
pprint.pprint(count)
```

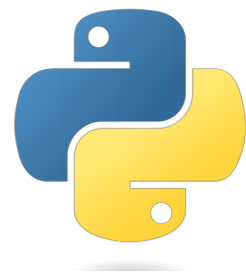
Sortie :

```
{ ' ': 13,  
  ',': 1,  
  '!': 1,  
  'A': 1,  
  'I': 1,  
  ...  
  't': 6,  
  'w': 2,  
  'y': 1 }
```



Dictionnaires et listes imbriqués

- Au fur et à mesure que vous modélisez des choses plus compliquées, il se peut que vous ayez besoin de dictionnaires et de listes qui contiennent d'autres dictionnaires et d'autres listes.
- Les listes sont utiles pour contenir une série ordonnée de valeurs, et les dictionnaires pour associer des clés à des valeurs.
- Par exemple, vous avez besoin d'écrire un programme qui utilise un dictionnaire contenant d'autres dictionnaires indiquant les articles apportés par les invités à un pique-nique.



Exemple : dictionnaire imbriqué

```
allGuests = {'Alice': {'apples': 5, 'pretzels': 12},  
             'Bob': {'ham sandwiches': 3, 'apples': 2},  
             'Carol': {'cups': 3, 'apple pies': 1}}
```

```
def totalBrought(guests, item):  
    numBrought = 0  
    for k, v in guests.items():  
        numBrought = numBrought + v.get(item, 0)  
  
    return numBrought  
  
print('Number of things being brought:')  
print(' - Apples      ' + str(totalBrought(allGuests, 'apples')))  
print(' - Cups        ' + str(totalBrought(allGuests, 'cups')))  
print(' - Cakes         ' + str(totalBrought(allGuests, 'cakes')))  
print(' - Ham Sandwiches ' + str(totalBrought(allGuests, 'ham sandwiches')))  
print(' - Apple Pies    ' + str(totalBrought(allGuests, 'apple pies')))
```

Sortie :

Number of things being brought:

- Apples 7
- Cups 3
- Cakes 0
- Ham Sandwiches 3
- Apple Pies 1



Opérations sur les dictionnaires

Type Python	Type abstrait	Opération	Exemple	Complexité
<code>dict</code>	Tableau associatif	Ajout d'un élément	<code>d[key] = val</code>	$O(1)$
<code>d={}</code>		Modification d'un élément	<code>d[key] = val</code>	$O(1)$
<code>len(d)</code>		Effacement d'un élément	<code>del d[key]</code>	$O(1)$
		Accès à un élément	<code>d[key]</code>	$O(1)$
		Recherche d'une clé	<code>key in d</code>	$O(1)$
		Recherche d'une valeur	<code>val in d.values()</code>	$O(n)$